为 Claude 构建 Skills 的完整指南

✦ Claude

目录

介绍 3

第一章：基础知识 4

第二章：规划与设计 7

第三章：测试与迭代 14

第四章：分发与共享 18

第五章：模式与故障排除 21

第六章：资源与参考 28

附录A：快速检查清单 30

附录B：YAML frontmatter 31

附录C：完整 Skill 示例 32

介绍

Skill 是一组指令——打包成一个简单的文件夹——用于教会 Claude 如何处理特定任务或工作流程。Skills 是定制 Claude 以满足您特定需求的最强大方式之一。与其在每次对话中重复解释您的偏好、流程和领域专业知识，不如通过 Skills 只教 Claude 一次，然后每次都能受益。

当您有可重复的工作流程时，Skills 非常强大：从规格说明生成前端设计、使用一致的方法进行研究、创建遵循团队风格指南的文档，或编排多步骤流程。它们与 Claude 的内置功能（如代码执行和文档创建）配合良好。对于构建 MCP 集成的人来说，Skills 增加了另一个强大的层级，帮助将原始工具访问转变为可靠、优化的工作流程。

本指南涵盖了构建有效 Skills 所需了解的一切——从规划和结构到测试和分发。无论您是为自己、团队还是社区构建 Skill，您都会在整个指南中找到实用的模式和真实的示例。

您将学到什么

- Skills 结构的技术要求和最佳实践
- 独立 Skills 和 MCP 增强工作流程的模式
- 我们在不同用例中看到的有效模式
- 如何测试、迭代和分发您的 Skills

适用人群

- 希望 Claude 始终如一地遵循特定工作流程的开发者
- 希望 Claude 遵循特定工作流程的高级用户
- 希望在组织内标准化 Claude 使用方式的团队

阅读本指南的两条路径

构建独立 Skills？ 重点关注"基础知识"、"规划与设计"以及第1-2类。

增强 MCP 集成？ "Skills + MCP"部分和第3类适合您。

两条路径共享相同的技术要求，但您可以选择与您的用例相关的内容。

您将从本指南中获得什么

阅读完本指南后，您将能够在一次坐下来的时间内构建一个功能完整的 Skill。使用 skill-creator 构建和测试您的第一个工作 Skill 大约需要 15-30 分钟。

让我们开始吧。

第一章

基础知识

基础知识

什么是 Skill?

Skill 是一个包含以下内容的文件夹：

- **SKILL.md**（必需）：带有 YAML frontmatter 的 Markdown 指令
- **scripts/**（可选）：可执行代码（Python、Bash 等）
- **references/**（可选）：按需加载的文档
- **assets/**（可选）：输出中使用的模板、字体、图标

核心设计原则

渐进式披露（Progressive Disclosure）

Skills 使用三级系统：

- 第一级（**YAML frontmatter**）：始终加载到 Claude 的 system prompt 中。提供足够的信息让 Claude 知道何时应该使用每个 Skill，而无需将所有内容加载到上下文中。
- 第二级（**SKILL.md** 正文）：当 Claude 认为 Skill 与当前任务相关时加载。包含完整的指令和指导。
- 第三级（链接文件）：Skill 目录中捆绑的附加文件，Claude 可以选择仅在需要时导航和发现。

这种渐进式披露最大限度地减少了 token 使用，同时保持专业化的专业知识。

可组合性（Composability）

Claude 可以同时加载多个 Skills。您的 Skill 应该能够与其他 Skills 良好配合，而不是假设它是唯一可用的功能。

可移植性（Portability）

Skills 在 Claude.ai、Claude Code 和 API 中的工作方式完全相同。创建一次 Skill，它就可以在所有平台上工作而无需修改，前提是环境支持 Skill 所需的任何依赖项。

对于 MCP 构建者：Skills + 连接器

> 💡 构建不使用 MCP 的独立 Skills？跳到"规划与设计"——您可以随时回来。

如果您已经有一个工作的 MCP 服务器，您已经完成了困难的部分。Skills 是在其之上的知识层——捕获您已经知道的工作流程和最佳实践，以便 Claude 可以一致地应用它们。

厨房类比

MCP 提供专业厨房：访问工具、食材和设备。

Skills 提供食谱：关于如何创造有价值东西的分步说明。

它们一起使用户能够完成复杂任务，而无需自己弄清楚每一步。

它们如何协同工作

MCP（连接性）	Skills（知识）
将 Claude 连接到您的服务	教 Claude 如何有效使用您的服务
提供实时数据访问和工具调用	捕获工作流程和最佳实践
Claude 能做什么	Claude 应该怎么做

为什么这对您的 MCP 用户很重要

没有 **Skills**：

- 用户连接您的 MCP 但不知道下一步该做什么
- 支持工单询问"如何使用您的集成做 X"
- 每次对话都从头开始
- 因为用户每次提示不同而导致结果不一致

有了 **Skills**：

- 预构建的工作流程在需要时自动激活
- 一致、可靠的工具使用
- 最佳实践嵌入每次交互中
- 降低您集成的学习曲线

第二章

规划与设计

规划与设计

从用例开始

在编写任何代码之前，确定您的 Skill 应该启用的 2-3 个具体用例。

好的用例定义

```
用例：项目 Sprint 规划
触发器：用户说"帮我规划这个 sprint"或"创建 sprint 任务"
步骤：
  1. 通过 MCP 获取当前项目状态
  2. 分析团队速度和容量
  3. 建议任务优先级
  4. 创建带有适当标签和估算的任务
结果：完整规划的 sprint，已创建任务
```

问问自己

- 用户想要完成什么？
- 这需要哪些多步骤工作流程？
- 需要哪些工具（内置或 MCP）？
- 应该嵌入哪些领域知识或最佳实践？

常见 Skill 用例类别

在 Anthropic，我们观察到三种常见用例：

类别1：文档和资产创建

用于：创建一致、高质量的输出，包括文档、演示文稿、应用程序、设计、代码等。

关键技术：

- 嵌入式风格指南和品牌标准
- 一致输出的模板结构
- 最终确定前的质量检查清单
- 无需外部工具——使用 Claude 的内置功能

类别2：工作流程自动化

用于：受益于一致方法论的多步骤流程，包括跨多个 MCP 服务器的协调。

关键技术：

- 带验证关卡的分步工作流程
- 常见结构的模板
- 内置审查和改进建议
- 迭代细化循环

类别3：MCP 增强

用于：增强 MCP 服务器提供的工具访问的工作流程指导。

关键技术：

- 按顺序协调多个 MCP 调用
- 嵌入领域专业知识
- 提供用户否则需要指定的上下文
- 常见 MCP 问题的错误处理

技术要求

文件结构

```
your-skill-name/
├── SKILL.md              # 必需
├── scripts/              # 可选
|   ├── process_data.py
|   └── validate.sh
├── references/           # 可选
|   └── api-guide.md
└── assets/               # 可选
    └── template.md
```

关键规则

SKILL.md 命名：

- 必须正好是 `SKILL.md`（区分大小写）
- 不接受变体（SKILL.MD、skill.md 等）

Skill 文件夹命名：

- 使用 kebab-case：`notion-project-setup` ✅
- 无空格：`Notion Project Setup` ❌
- 无下划线：`notion_project_setup` ❌

YAML frontmatter

YAML frontmatter 是 Claude 决定是否加载您的 Skill 的方式。

最小必需格式

```
---
name: your-skill-name
description: What it does. Use when user asks to [specific
phrases].
---
```

字段要求

name（必需）：仅 kebab-case，无空格或大写，应与文件夹名称匹配

description（必需）：必须包含 Skill 做什么和何时使用它，少于 1024 个字符

第三章

测试与迭代

测试与迭代

根据您的需求，可以在不同的严格程度上测试 Skills：

- 在 **Claude.ai** 中手动测试 - 直接运行查询并观察行为。快速迭代，无需设置。
- 在 **Claude Code** 中脚本化测试 - 自动化测试用例，以便在更改时进行可重复验证。
- 通过 **Skills API** 进行程序化测试 - 构建针对定义的测试集系统运行的评估套件。

> 专业提示：在扩展之前迭代单个任务
>
> 我们发现，最有效的 Skill 创建者会在单个具有挑战性的任务上迭代，直到 Claude 成功，然后将获胜的方法提取到 Skill 中。

推荐的测试方法

1. 触发测试

目标：确保您的 Skill 在正确的时间加载。

- ✅ 在明显任务上触发
- ✅ 在改述请求上触发
- ❌ 不在无关主题上触发

2. 功能测试

目标：验证 Skill 产生正确的输出。

- 生成有效输出
- API 调用成功
- 错误处理有效
- 边缘情况已覆盖

3. 性能比较

目标：证明 Skill 相比基线改善了结果。

无 Skill	有 Skill
用户每次提供说明	自动工作流程执行
15 次来回消息	仅 2 个澄清问题
3 次失败的 API 调用	0 次失败的 API 调用
消耗 12,000 tokens	消耗 6,000 tokens

第四章

分发与共享

分发与共享

Skills 使您的 MCP 集成更完整。当用户比较连接器时，带有 Skills 的连接器提供更快的价值路径。

当前分发模型

个人用户如何获取 Skills：

1. 下载 Skill 文件夹
2. 压缩文件夹（如需要）
3. 通过 Settings > Capabilities > Skills 上传到 Claude.ai
4. 或放置在 Claude Code skills 目录中

开放标准

我们已将 Agent Skills 作为开放标准发布。与 MCP 一样，我们相信 Skills 应该可以跨工具和平台移植。

当今推荐的方法

1. **托管在 GitHub 上** - 公共仓库，清晰的 README
2. **在 MCP 文档中记录** - 链接到 Skills，解释价值
3. **创建安装指南** - 分步说明

第五章

模式与故障排除

模式与故障排除

常用模式

Pattern 1：顺序工作流程编排

使用场景：您的用户需要按特定顺序执行多步骤流程。

关键技术：显式步骤排序、步骤之间的依赖关系、每个阶段的验证、失败的回滚指令。

Pattern 2：多 MCP 协调

使用场景：工作流程跨越多个服务。

关键技术：清晰的阶段分离、MCP 之间的数据传递、集中错误处理。

Pattern 3：迭代细化

使用场景：输出质量随迭代而提高。

关键技术：显式质量标准、验证脚本、知道何时停止迭代。

故障排除

Skill 不触发

症状：Skill 从不自动加载

修复：修改您的 description 字段，添加更多触发短语

Skill 触发太频繁

症状：Skill 为无关查询加载

解决方案：添加否定触发器，更具体

指令未被遵循

常见原因：

- 指令太冗长 - 保持简洁
- 指令被埋没 - 将关键指令放在顶部
- 语言模糊 - 使用具体的、可操作的语言

第六章

资源与参考

资源与参考

官方文档

- Best Practices Guide
- Skills Documentation
- API Reference
- MCP Documentation

示例 Skills

公共 **Skills** 仓库：GitHub: anthropics/skills

工具和实用程序

skill-creator skill：

- 内置于 Claude.ai，可用于 Claude Code
- 可从描述生成 Skills
- 审查并提供建议

获取支持

技术问题：Claude Developers Discord

Bug 报告：GitHub Issues: anthropics/skills/issues